

Лабораторная работа №2

Проектирование модели данных и серверной части

Цель работы:

Освоить проектирование модели данных, разработку алгоритма планирования и первичное проектирование серверной логики веб-приложения.

Формат отчётности:

Документ в формате .docx (Word). Отчёт сдаётся в электронном виде. Все этапы оформляются в одном документе.

Этап 1. Проектирование базы данных

1. Построить ER-диаграмму предметной области.

- Минимальное количество сущностей — **4–5**.
- Диаграмма должна включать:
 - наименования сущностей (на русском языке);
 - атрибуты (на русском языке);
 - первичные ключи;
 - внешние ключи и связи (с указанием типа связи: 1:1, 1:M, M:M).
- Допускаемые инструменты: [Draw.io](https://draw.io), ERwin, Visio, Lucidchart и т.д.

2. Разработать схему базы данных (логический уровень).

- Представить таблицы с полями, типами, ограничениями (NOT NULL, UNIQUE и т.п.) и связями между ними.

3. Составить SQL-скрипты создания таблиц.

- Скрипты должны быть **работоспособными**, полностью отображающими спроектированную модель данных.

4. Привести описание схемы в тексте отчёта.

- ER-диаграмма.
- Описание каждой таблицы: назначение, ключевые поля, связи.
- Листинг SQL-скриптов.

Этап 2. Реализация серверной части (базовый каркас)

1. Настроить проект backend-приложения.

- Выбранный стек: Java + Spring Boot / Python + FastAPI / Node.js и т.д.
- Создать структуру проекта.

2. Реализовать модели данных (entities).

- Классы/структуры, соответствующие таблицам БД.
- Описать поля и аннотации/отображение (JPA, SQLAlchemy и т.п.).

3. Реализовать слой доступа к данным (DAO/репозитории).

- Базовые CRUD-операции (create, read, update, delete) для основных сущностей.
- Подключение к БД (на уровне конфигурации).

Этап 3. Проектирование алгоритма планирования

3.1. Содержательная постановка задачи

- Описать **входные данные** алгоритма (что поступает на вход: список заказов, перечень ресурсов, временные ограничения и т.д.).
- Описать **выходные данные** (результат работы алгоритма: расписание, назначения, график).
- Сформулировать **цель алгоритма** (например: минимизировать простои, выполнить все заказы в срок, равномерно загрузить ресурсы).

3.2. Разработка пошагового описания алгоритма

Алгоритм должен быть описан **на естественном языке** и в виде блок-схем (в соответствии с ГОСТ) с чёткой пошаговой логикой.

3.2.1 Требования к текстовому описанию:

- Последовательность шагов нумеруется.
- Используются логические конструкции: «для каждого ...», «если ... то ... иначе ...», «пока ...».

- Указываются критерии выбора (например: «выбрать первый доступный ресурс», «выбрать ресурс с наименьшей загрузкой»).
- Учитываются ограничения предметной области (время работы, совместимость, последовательность операций).

Пример фрагмента алгоритма (неполный, для иллюстрации стиля)

Тема: Планирование работы мобильных бригад

Входные данные:

- список заявок (номер заявки, адрес объекта, плановая длительность работ, желаемое время);
- список бригад (идентификатор, квалификация, график работы);
- календарь на 7 дней с часовыми слотами.

Выходные данные:

- расписание назначений бригад на заявки.

Алгоритм:

1. Отсортировать все заявки по желаемому времени выполнения (от ранних к поздним).
2. Для каждой заявки из отсортированного списка:
 - 2.1. Определить минимальную требуемую квалификацию для выполнения заявки.
 - 2.2. Выбрать из всех бригад только те, у которых квалификация не ниже требуемой.
 - 2.3. Отсортировать доступные бригады по количеству уже назначенных часов (по возрастанию).
 - 2.4. Для каждой бригады из отсортированного списка:
 - 2.4.1. Найти ближайший свободный временной слот, длительность которого не меньше длительности заявки.
 - 2.4.2. Если такой слот найден — назначить заявку на этот слот, зафиксировать занятость бригады, перейти к следующей заявке.
 - 2.4.3. Если слот не найден — перейти к следующей бригаде.
 - 2.5. Если ни одна бригада не найдена — заявка помечается как «не назначена».
3. Сформировать выходной отчёт — расписание в хронологическом порядке.

3.2.2 Требования к блок-схемам алгоритма

При выполнении алгоритма в виде блок-схемы необходимо руководствоваться **ГОСТ 19.701–90 (ИСО 5807–85) «Схемы алгоритмов, программ, данных и систем»**: использовать стандартные графические символы (пуск/останов, процесс, решение, ввод/вывод, модификация, соединитель и др.) и соблюдать направления потоков; допускается выполнение блок-схемы от руки с последующим сканированием, но с обязательным применением утверждённых обозначений.

Этап 4. Проектирование REST API

4.1. Перечень проектируемых эндпоинтов

Для **каждого** ключевого пользовательского сценария определить:

- **HTTP-метод** (GET, POST, PUT, DELETE).
- **URL шаблон** (например: /api/tasks, /api/schedule/{id}).
- **Назначение** (что делает данный эндпоинт).
- **Входные параметры** (query-параметры, тело запроса, путь).
- **Формат ответа** (краткое описание, что возвращается).
- **Привязка к модели данных** (какие таблицы/сущности затрагиваются).
- **Привязка к экрану интерфейса** (на каком экране и для какого действия пользователя этот эндпоинт будет вызываться).

Минимальное количество эндпоинтов — 5.

Пример оформления (фрагмент):

Метод	URL	Назначение	Входные данные	Выходные данные	Привязка к БД	Привязка к экрану
GET	/api/brigades	Получить список всех бригад	—	Массив объектов {id, name, qualification}	Таблица brigades	Экран «Планирование» — выпадающий список бригад
POST	/api/assignments	Назначить бригаду на заявку	{request_id, brigade_id, start_time}	{assignment_id, status}	Таблицы assignments, requests, brigades	Экран «Назначение» — кнопка «Назначить»
GET	/api/schedule/{date}	Получить расписание на дату	date (строка)	Массив назначений с деталями	Таблицы assignments, requests, brigades (соединение)	Экран «Календарь» — отображение занятости